

DELHI TECHNOLOGICAL UNIVERSITY
(Formerly Delhi College of Engineering) Shahbad
Daulatpur, Bawana Road, Delhi-110042

**DEPARTMENT OF COMPUTER SCIENCE
AND ENGINEERING**



Machine Learning Lab File
CO303

Submitted To:

Assistant Prof. Gull Kaur
Department of Computer
Science and Engineering

Submitted By:

Krish Bansal
24/CS/248

INDEX

SNo.	Aim	Date	Page No.	Signature
1	To implement and evaluate the K-Nearest Neighbors (KNN) classification algorithm on the Iris dataset.			
2	To formulate a machine learning problem, inspect and assess the health of a dataset, separate features and target variables, split the dataset into training and testing sets, and establish a dummy baseline model for evaluating the usefulness of a machine learning			
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				

EXPERIMENT – 2

AIM:

To formulate a machine learning problem, inspect and assess the health of a dataset, separate features and target variables, split the dataset into training and testing sets, and establish a dummy baseline model for evaluating the usefulness of a machine learning

THEORY:

Binary Classification

Binary classification is a supervised machine learning problem in which the target variable contains two possible classes.

In this experiment, the objective is to predict whether a passenger on the Titanic **survived or did not survive**.

The target variable is:

- **0:** Did not survive
- **1:** Survived

The Titanic dataset contains information about passengers such as passenger class, sex, age, number of siblings or spouses, number of parents or children, fare, and port of embarkation.

Naive Bayes

Naive Bayes is a supervised classification algorithm based on **Bayes' Theorem**.

Bayes' theorem is given by:

$$P(C|X) = P(X|C)P(C) / P(X)$$

where:

- $P(C|X)$ is the posterior probability of class C given the input features.
- $P(X|C)$ is the likelihood of observing the input features given class C.
- $P(C)$ is the prior probability of class C.
- $P(X)$ is the probability of observing the input features.

The algorithm is called "naive" because it assumes that the features are conditionally independent given the class.

For continuous numerical features, **Gaussian Naive Bayes** assumes that the features follow a Gaussian distribution.

Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression.

For classification, SVM attempts to find a decision boundary or **hyperplane** that separates different classes while maximizing the margin between them.

The observations closest to the decision boundary are called **support vectors**.

SVM can use kernel functions to handle non-linear relationships. In this experiment, an **RBF (Radial Basis Function) kernel** is used.

The RBF kernel is given by:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

The SVM model used in this experiment has:

- **kernel = 'rbf'**
- **C = 1.0**
- **gamma = 'scale'**
- **random_state = 67**

Dummy Classifier

A Dummy Classifier is a simple baseline model that does not attempt to learn meaningful relationships between features and the target.

It is useful for determining whether a machine learning model performs better than a simple baseline.

Two dummy classifier strategies are used:

1. **Most Frequent:** Always predicts the most frequent class.
2. **Stratified:** Generates predictions according to the class distribution observed in the training data.

Feature Preprocessing

The Titanic dataset contains categorical and numerical features.

The following preprocessing operations are performed:

- PassengerId, Name, Ticket, and Cabin are removed.
- Sex is converted into numerical values using LabelEncoder.
- Embarked is converted into numerical values using LabelEncoder.
- Missing values in Age are handled using median imputation.
- Age and Fare are standardized using StandardScaler.

The dataset contains **891 rows and 12 columns initially**. After removing four columns, the working dataset contains **891 rows and 8 columns**. The Age column initially contains **177 missing values**.

The data is divided into **70% training data and 30% testing data** using `random_state=67`. The resulting test set contains **268 samples**.

CODE:

```
import kagglehub

path = kagglehub.dataset_download("yasserh/titanic-dataset")

print("Path to dataset files:", path)

import pandas as pd

df = pd.read_csv(

    "/kaggle/input/titanic-dataset/Titanic-Dataset.csv"

)

df.head()
```

```
df.shape
df.columns
df.drop(
    ['PassengerId', 'Name', 'Ticket', 'Cabin'],
    axis=1,
    inplace=True
)
df.head()
import seaborn as sns
import matplotlib.pyplot as plt
plt.hist(
    df['Age'],
    bins=30,
    color='skyblue',
    edgecolor='black'
)
sns.boxplot(df['Age'])
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex'])
df['Embarked'].nunique()
df['Embarked'] = le.fit_transform(
    df['Embarked']
)
df.head()
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
print(df.isna().sum())
df.shape
```

```

import numpy as np
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(
    missing_values=np.nan,
    strategy='median'
)
X = df.drop('Survived', axis=1)
y = df['Survived']
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=67
)
X_train[['Age', 'Fare']] = sc.fit_transform(
    X_train[['Age', 'Fare']]
)
X_test[['Age', 'Fare']] = sc.transform(
    X_test[['Age', 'Fare']]
)
X_train[['Age']] = imputer.fit_transform(
    X_train[['Age']]
)
X_test[['Age']] = imputer.transform(
    X_test[['Age']]
)
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (

```

```

        accuracy_score,
        classification_report,
        confusion_matrix
    )
    dcf = DummyClassifier(
        strategy="most_frequent"
    )
    dcf.fit(X_train, y_train)
    y_pred = dcf.predict(X_test)
    accuracy = accuracy_score(
        y_test,
        y_pred
    )
    print(f"Accuracy: {accuracy:.2f}")
    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))
    dcf2 = DummyClassifier(
        strategy="stratified"
    )
    dcf2.fit(X_train, y_train)
    y_pred = dcf2.predict(X_test)
    accuracy = accuracy_score(
        y_test,
        y_pred
    )
    print(f"Accuracy: {accuracy:.2f}")
    print("\nConfusion Matrix:")

```



```
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
from sklearn.naive_bayes import GaussianNB
bayes = GaussianNB()
bayes.fit(
    X_train,
    y_train
)
y_pred = bayes.predict(X_test)
accuracy = accuracy_score(
    y_test,
    y_pred
)
print(f"Accuracy: {accuracy:.2f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
from sklearn import svm
svm_model = svm.SVC(
    kernel='rbf',
    C=1.0,
    gamma='scale',
    random_state=67
)
svm_model.fit(
    X_train,
    y_train
```

```
)
predictions = svm_model.predict(
    X_test
)
accuracy = accuracy_score(
    y_test,
    predictions
)
print(f"Accuracy: {accuracy:.2f}")
print("\nConfusion Matrix:")
print(confusion_matrix(
    y_test,
    predictions
))
print("\nClassification Report:")
print(classification_report(
    y_test,
    predictions
))
```

OUTPUT:

The Titanic dataset initially contained:

- **Total samples:** 891
- **Total columns:** 12
- **Target variable:** Survived
- **Initial missing values in Age:** 177

After removing PassengerId, Name, Ticket, and Cabin, the dataset contained:

- **Samples:** 891
- **Columns:** 8

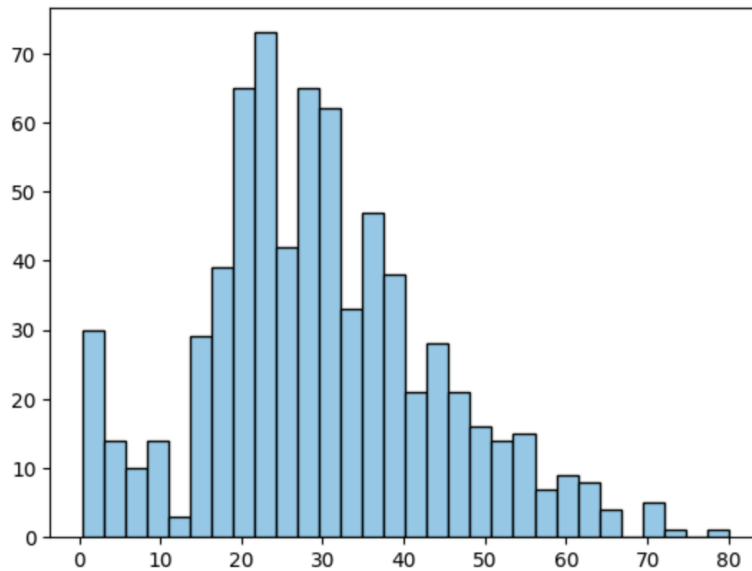
The final features were:

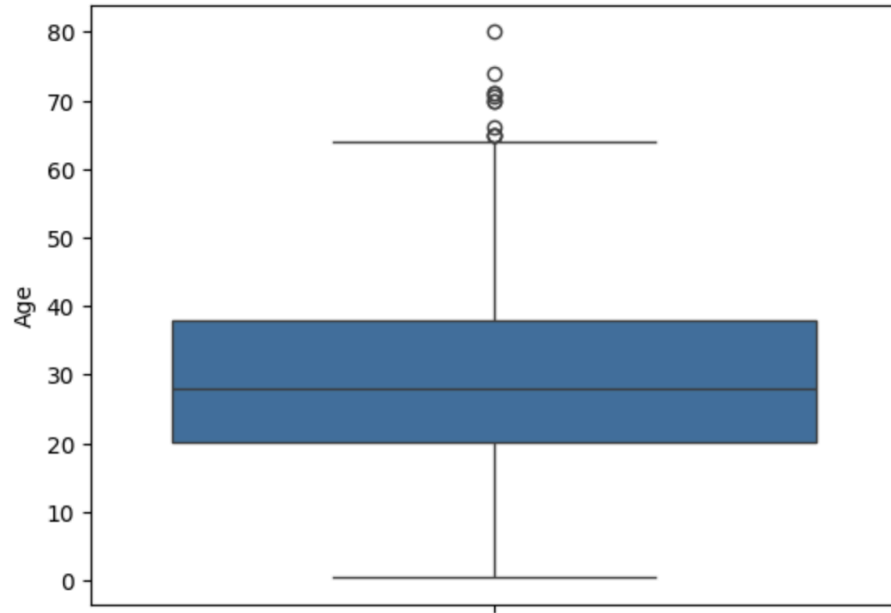
- Pclass
- Sex
- Age
- SibSp
- Parch
- Fare
- Embarked

The target variable was:

- Survived

The train-test split produced **70% training data and 30% testing data**, with **268 test samples**.





Dummy Classifier – Most Frequent

The most-frequent dummy classifier achieved:

Accuracy: 0.60

Confusion Matrix:

[[162 0]

[106 0]]

Classification Report:

Class	Precision	Recall	F1-Score	Support
0	0.60	1.00	0.75	162
1	0.00	0.00	0.00	106
Accuracy			0.60	268

The classifier always predicted the most frequent class, resulting in zero predictions for class 1.

Dummy Classifier – Stratified

The stratified dummy classifier achieved:

Accuracy: 0.54

Confusion Matrix:

```
[[104 58]
```

```
[ 66 40]]
```

Classification Report:

Class	Precision	Recall	F1-Score	Support
0	0.61	0.64	0.63	162
1	0.41	0.38	0.39	106
Accuracy			0.54	268

The stratified classifier provides a more useful baseline because it produces predictions according to the observed class distribution.

Gaussian Naive Bayes

The Gaussian Naive Bayes classifier achieved:

Testing Accuracy: 0.79

Confusion Matrix:

```
[[132 30]
```

```
[ 26 80]]
```

Classification Report:

Class	Precision	Recall	F1-Score	Support
0	0.84	0.81	0.82	162
1	0.73	0.75	0.74	106
Accuracy			0.79	268
Macro Average	0.78	0.78	0.78	268
Weighted Average	0.79	0.79	0.79	268

The Gaussian Naive Bayes model significantly outperformed both dummy classifier baselines.

Support Vector Machine

The SVM model was implemented using an **RBF kernel** with C=1.0 and gamma='scale'.

The model achieved:

Testing Accuracy: 0.83

Confusion Matrix:

```
[[145 17]
```

```
[ 29 77]]
```

Classification Report:

Class	Precision	Recall	F1-Score	Support
0	0.83	0.90	0.86	162
1	0.82	0.73	0.77	106
Accuracy			0.83	268
Macro Average	0.83	0.81	0.82	268
Weighted Average	0.83	0.83	0.83	268

The SVM achieved the highest accuracy among the models tested in this experiment.

Model Comparison

Model	Testing Accuracy
Dummy – Most Frequent	0.60
Dummy – Stratified	0.54
Gaussian Naive Bayes	0.79
Support Vector Machine (RBF)	0.83

The results show that both machine learning models performed substantially better than the dummy baselines. **SVM achieved the highest testing accuracy of 83%**, followed by **Gaussian Naive Bayes with 79%**.

The SVM model correctly classified 145 non-survivors and 77 survivors, while Gaussian Naive Bayes correctly classified 132 non-survivors and 80 survivors.

LEARNING OUTCOME:

1. Formulate a **binary classification problem** using the Titanic dataset.
2. Load and inspect a real-world dataset using Pandas.
3. Analyze the shape, columns, and missing values of a dataset.
4. Remove irrelevant columns such as PassengerId, Name, Ticket, and Cabin.
5. Encode categorical variables using **LabelEncoder**.
6. Handle missing values using **median imputation**.
7. Apply **StandardScaler** to numerical features such as Age and Fare.
8. Divide the dataset into training and testing sets using train_test_split().
9. Establish baseline performance using **DummyClassifier**.
10. Understand and implement the **Gaussian Naive Bayes** classification algorithm.
11. Understand and implement **Support Vector Machine** classification.
12. Understand the use of the **RBF kernel** in SVM.
13. Evaluate classification models using **accuracy, confusion matrix, precision, recall, and F1-score**.
14. Compare machine learning models against simple baseline classifiers.
15. Observe that the **SVM model achieved the best performance with 83% testing accuracy** on the given test split.